

UNIVERSIDAD NACIONAL DE LOJA

FACULTAD DE LA ENERGÍA, LAS INDUSTRIAS
Y LOS RECURSOS NATURALES NO RENOVABLES

Carrera de Computación



Universidad
Nacional
de Loja

Especificación de Requisitos y Análisis de Gestión de Memoria (V8): Indexación Jerárquica con GreenBST para Scraping Agrícola

Autores:

Isaac Alejandro Vire Poma
Noelia Alexandra Bustán Hurtado
Lucas Sebastián Orellana Martínez
Danny Fernando Beltran Alulima
Axel Joao Jiménez Palta

Grupo 09

Tercer Ciclo Paralelo A

7 de octubre de 2026

Índice

1. Introducción	2
1.1. Propósito del Proyecto	2
1.2. Alcance y Conexión Teórica	2
1.3. Fundamentos Teóricos: Call Stack	2
1.4. Fundamentos Teóricos: Memory Heap y Punteros	2
2. Descripción General	3
2.1. Perspectiva del Producto: Ejecución en V8	3
2.2. Utilidad en la Ingeniería de Software	3
3. Requisitos Específicos y Restricciones	4
3.1. Riesgos relacionados con la Memoria (Stack y Heap)	4
3.2. Requisitos Funcionales del Sistema	5
3.3. Requisitos No Funcionales del Sistema	6
4. Conclusiones	6
5. Referencias	7

1. Introducción

1.1. Propósito del Proyecto

El propósito de este proyecto es desarrollar un prototipo que recopile, organice e indexe los precios de productos agrícolas de las ferias de Loja, para ello el proyecto propone resolverlo con un árbol binario de búsqueda auto-balanceado, pues a medida que un sistema recibe datos de forma continua, buscar en un arreglo lineal se vuelve cada vez más lento, porque con cada consulta debe revisar los registros uno por uno. Una aplicación web mostrará de forma visual la diferencia de latencia entre ambos enfoques. Con ello, el trabajo busca, por un lado, aplicar y comprender estructuras de datos y análisis de complejidad en un caso concreto y, por otro, ofrecer una base que pueda servir a productores y consumidores para consultar con rapidez la información de precios del mercado local.

1.2. Alcance y Conexión Teórica

El presente documento especifica los requisitos funcionales y no funcionales para el desarrollo de este sistema. La teoría sobre el motor V8 que se presentará a continuación y los requisitos aquí definidos nos ayudarán directamente con el desarrollo de nuestro proyecto, brindándonos las bases teóricas y técnicas para gestionar la memoria de manera eficiente y prevenir desbordamientos durante la indexación masiva de datos.

1.3. Fundamentos Teóricos: Call Stack

El Call Stack o pila de llamadas es una estructura de datos que utiliza el motor de ejecución para controlar las funciones que se encuentran activas durante la ejecución de un programa. Su funcionamiento se basa en el principio LIFO (Last-In, First-Out), es decir, la última función que ingresa a la pila es la primera en salir. En el caso del motor V8 de JavaScript, cada vez que se invoca una función se incorpora un nuevo marco de ejecución al Call Stack y, cuando la función termina, dicho marco es retirado de la pila [1].

1.4. Fundamentos Teóricos: Memory Heap y Punteros

El Memory Heap o montón de memoria es una región extensa y no estructurada de la memoria RAM que utiliza el motor V8 para almacenar datos de tamaño dinámico y de vida útil prolongada. A diferencia del Call Stack, que gestiona llamadas a funciones de forma secuencial y estricta, el Heap actúa como un almacén dinámico donde residen los objetos, instancias de clases, arreglos, funciones y cierres (closures) [2]. Dentro de la arquitectura de V8, el Memory Heap se organiza internamente en diferentes zonas o 'espacios' para optimizar la gestión y limpieza de la memoria:

- **New Space:** Zona donde se asignan inicialmente la mayoría de los objetos pequeños y efímeros.
- **Old Space:** Espacio donde se transfieren los objetos que sobreviven a varios ciclos de recolección de basura en el New Space.

- **Large Object Space:** Reservado para objetos cuyo tamaño supera los límites de las zonas convencionales, evitando su movimiento constante durante la desfragmentación.

Dado que las variables alojadas en el Heap no se destruyen automáticamente al finalizar una función, su acceso se realiza mediante punteros o referencias almacenadas en la pila (Call Stack) [3].

2. Descripción General

2.1. Perspectiva del Producto: Ejecución en V8

El Call Stack permite mantener el orden en que se ejecutan las funciones y conocer qué función se encuentra activa en cada momento. Por ejemplo, si una función A llama a una función B, primero se incorpora A a la pila y posteriormente B. Cuando B finaliza, se retira de la pila y la ejecución continúa en A. Este mecanismo permite controlar de manera organizada las llamadas y retornos de las funciones [1]. Además, el Call Stack mantiene una relación directa con el Memory Heap. Mientras la pila se utiliza para controlar la ejecución y almacenar información asociada a los marcos de las funciones, el Heap es utilizado por V8 para almacenar objetos y otros datos dinámicos. Las referencias a estos objetos permiten relacionar ambas áreas de memoria durante la ejecución del programa.

El funcionamiento del motor V8 depende de una sinergia constante entre el Call Stack, el Memory Heap y el Garbage Collector:

- **Vínculo por Referencias:** Durante la ejecución de una función en el Call Stack, las variables locales guardan únicamente direcciones de memoria o punteros que apuntan a la ubicación física de los objetos dentro del Memory Heap.
- **Ciclo de Liberación:** Cuando la función finaliza y su marco de ejecución es retirado del Call Stack, los punteros desaparecen. Si el objeto en el Heap se queda sin ninguna referencia que lo alcance desde la pila, pasa a ser candidato para ser eliminado.
- **Recolección Automática:** El Garbage Collector de V8 recorre periódicamente el Heap identificando y destruyendo los objetos huérfanos, devolviendo el espacio libre al sistema para futuras asignaciones.

Esta división garantiza que JavaScript mantenga una ejecución organizada para el control del flujo del programa en la pila, mientras dispone de un espacio amplio y flexible en el montón para alojar la información física del sistema. En el motor V8, el proceso de ejecución puede comprenderse mediante la interacción entre estas dos áreas. Cuando una función termina, su marco es retirado del Call Stack, pero los objetos en el Heap permanecen mientras sean accesibles [1].

2.2. Utilidad en la Ingeniería de Software

El conocimiento del Call Stack resulta importante para comprender el comportamiento de un programa y facilitar la identificación de errores durante su ejecución. Al observar la secuencia de funciones activas, es posible determinar qué operaciones se estaban realizando cuando ocurrió un problema. El análisis del Stack también permite comprender mejor el rendimiento de las aplicaciones. Una cantidad excesiva de llamadas anidadas puede

aumentar el uso de la pila y afectar el funcionamiento del programa. Por esta razón, comprender la forma en que las funciones ingresan y salen del Call Stack constituye una base importante para el desarrollo y la depuración de software [1].

El Memory Heap es fundamental para permitir que las aplicaciones modernas de JavaScript manejen estructuras de datos complejas y de tamaño variable en tiempo de ejecución. Permite la existencia de la Programación Orientada a Objetos (POO) y la persistencia de estados globales en aplicaciones web y servidores Node.js [4]. Comprender el funcionamiento del Heap es vital para la optimización de recursos y la sostenibilidad del software:

- **Gestión de la Complejidad Espacial:** Permite determinar cómo el diseño de software impacta la huella física de memoria al procesar volúmenes masivos de información [4].
- **Eficiencia en Estructuras de Datos:** Conocer el Heap ayuda a elegir entre abstracciones de alto nivel o estructuras continuas fuertemente tipadas TypedArrays, optimizando la transferencia de datos directamente en el procesador.
- **Ciclo de Vida de Objetos:** Permite diseñar algoritmos que reutilicen memoria y eviten la saturación anticipada del entorno de ejecución [4].

3. Requisitos Específicos y Restricciones

3.1. Riesgos relacionados con la Memoria (Stack y Heap)

El uso incorrecto de los recursos asociados con la ejecución puede producir diferentes problemas en una aplicación. En el caso del Call Stack, uno de los principales riesgos es el Stack Overflow, que puede aparecer cuando las llamadas a funciones aumentan excesivamente. Un ejemplo común es una función recursiva que continúa llamándose a sí misma sin alcanzar una condición de finalización. En JavaScript, este tipo de situación puede generar errores relacionados con el tamaño máximo permitido para el Call Stack.

Por otra parte, los problemas de memoria pueden estar relacionados con el Heap. En V8, los objetos y datos dinámicos se almacenan en esta zona de memoria. Cuando los objetos dejan de ser accesibles, el recolector de basura puede identificar y recuperar el espacio que ocupan. Sin embargo, una gestión inadecuada de las referencias puede contribuir a problemas de consumo de memoria y, eventualmente, producir errores de falta de memoria [1].

A diferencia de la pila que puede sufrir de Stack Overflow, los riesgos principales asociados al Memory Heap están vinculados al agotamiento del espacio dinámico y al impacto en el rendimiento del procesador:

- **Fugas de Memoria (Memory Leaks):** Ocurren cuando un objeto alojado en el Heap ya no es necesario para la lógica del programa, pero sigue conservando referencias activas desde la pila o variables globales. Esto impide que el recolector de basura libere dicho espacio, acumulando memoria consumida hasta provocar el colapso del proceso [5].
- **Pausas por Garbage Collection (Stop-The-World):** Cuando el Heap acumula demasiados objetos efímeros o desorganizados, el mecanismo de limpieza de V8 (como el recolector Orinoco) debe pausar momentáneamente la ejecución del programa

para rastrear referencias y liberar memoria, lo que genera latencia o congelamiento en la interfaz [5].

- **Fragmentación de Memoria y Overhead de Metadatos:** La creación masiva de objetos individuales genera cabeceras internas y envoltorios adicionales multiplicando el consumo real de RAM frente al valor numérico almacenado [5].

3.2. Requisitos Funcionales del Sistema

A continuación, se detallan los requisitos funcionales del Grupo 09:

ID	Requisito funcional
RF-01	El sistema debe incluir un bot de scraping que extraiga producto, feria, precio y fecha de una página web.
RF-02	La fuente del scraping debe ser configurable, para poder trabajar, aunque el sitio real de las ferias de Loja no esté disponible.
RF-03	El sistema debe realizar una limpieza básica de los datos extraídos: convertir el precio a número, normalizar el nombre del producto (minúsculas y sin tildes) y descartar registros incompletos.
RF-04	El sistema debe implementar la clase GreenBST en TypeScript (árbol AVL) con el nombre del producto como clave, donde cada nodo guarda la lista de registros de ese producto (feria, precio y fecha). Debe permitir insertar registros rebalanceando el árbol con rotaciones simples y dobles.
RF-05	El GreenBST debe permitir buscar un producto por su nombre y devolver todos sus registros.
RF-06	El GreenBST debe permitir eliminar un producto y volver a balancear el árbol.
RF-07	El GreenBST debe permitir recorrer los productos en orden alfabético.
RF-08	El sistema debe implementar una Tabla Hash sencilla que indexe los registros por feria.
RF-09	El sistema debe permitir ampliar el conjunto de datos obtenido por scraping hasta una cantidad de registros lo suficientemente amplia como para que las pruebas de rendimiento puedan cumplir su propósito.
RF-10	El sistema debe mantener un arreglo lineal con los mismos datos que el GreenBST, como estructura de referencia para la comparación.
RF-11	El sistema debe emular la ingesta continua, cargando lotes de datos en el GreenBST y en el arreglo lineal a intervalos regulares mientras se realizan las búsquedas.
RF-12	El sistema debe medir la latencia de búsqueda del GreenBST frente al arreglo lineal con distintos tamaños de datos y guardar los resultados.
RF-13	La aplicación web en Next.js debe permitir buscar un producto, filtrando por feria sobre sus registros y mostrando un mensaje de "sin resultados" si no existe, y mostrar un gráfico comparativo de latencia entre el GreenBST y la búsqueda lineal.

3.3. Requisitos No Funcionales del Sistema

Se establecen los siguientes requisitos no funcionales para garantizar el rendimiento, modularidad y seguridad del sistema:

ID	Requisito no funcional
RNF-01	Las operaciones de inserción, búsqueda y eliminación del GreenBST deben tener complejidad $O(\log n)$ y las de la Tabla Hash $O(1)$ en promedio. El sistema debe indexar al menos 100 000 registros sin una degradación que contradiga la complejidad teórica.
RNF-02	Los nodos y registros se almacenarán en el Memory Heap de V8 y su consumo por nodo debe estar documentado, creciendo de forma lineal con el número de registros. Los recorridos del árbol no deben provocar desbordamientos de pila (se admiten implementaciones iterativas).
RNF-03	El GreenBST debe mantener sus invariantes de ordenamiento y balance tras cualquier secuencia de operaciones, verificable mediante una función de validación y pruebas unitarias del árbol y de la Tabla Hash.
RNF-04	Las pruebas de latencia deben usar los mismos datos y las mismas búsquedas para el GreenBST y el arreglo lineal, repetirse al menos 30 veces y reportar la mediana.
RNF-05	El bot de scraping debe fallar de forma controlada ante errores de red o cambios en la página fuente, sin corromper el índice.
RNF-06	El bot debe respetar el archivo robots.txt y los términos de uso de las fuentes, limitar la tasa de peticiones (rate limiting) e identificarse con un User-Agent descriptivo.
RNF-07	Los datos recolectados deben limitarse a información pública de precios, sin datos personales.
RNF-08	El código debe estar en TypeScript y organizado en módulos separados para las estructuras de datos, el scraping y la interfaz.
RNF-09	El proyecto debe incluir un README (instalación y uso) y comentarios TSDoc en las clases públicas.
RNF-10	Los gráficos deben ser legibles, con ejes, unidades y leyendas claras, y mostrar mensajes comprensibles ante errores o falta de datos.
RNF-11	El sistema debe ejecutarse en Node.js LTS y reproducirse mediante un package.json con versiones fijadas.

4. Conclusiones

- **Optimización de Memoria en V8:** La comprensión de la arquitectura del motor V8, específicamente la separación entre el Call Stack (gestión estricta LIFO para la ejecución) y el Memory Heap (almacenamiento dinámico de objetos), es esencial para diseñar algoritmos que eviten cuellos de botella, *Memory Leaks* y pausas prolongadas del *Garbage Collector* en aplicaciones Node.js.
- **Eficiencia Algorítmica con GreenBST:** La implementación de estructuras de datos jerárquicas y auto-balanceadas, como el árbol AVL (GreenBST), proporciona una ventaja computacional crítica frente a los arreglos lineales, permitiendo indexar

y buscar masivamente registros agrícolas con una latencia mínima y garantizando una complejidad logarítmica sostenida.

- **Conexión con el Desarrollo del Proyecto:** La teoría que se ha abordado sobre la estructura interna de JavaScript, unida a los requisitos aquí presentes, nos ayudarán significativamente con el desarrollo de nuestro proyecto. Al entender cómo funciona la asignación de memoria en V8, podremos garantizar que la indexación de nuestro árbol GreenBST no sature el entorno, asegurando que el bot de scraping funcione de manera eficiente y escalable.

5. Referencias

- 1 K. Ravi, “Under the Hood: How V8 Manages the Stack and the Heap,” DEV Community, Feb. 11, 2024. [En línea]. Disponible en: https://dev.to/kiran_ravi_092a2cfc60389/under-the-hood-how-v8-manages-the-stack-and-the-heap-nho
- 2 M. Collina. “Boost Node.js with V8 GC Optimization”. Platformatic Blog. Accedido el 8 de octubre de 2026. [En línea]. Disponible: <https://blog.platformatic.dev/optimizing-nodejs-performance-v8-memory-management-and-gc-tuning>
- 3 “Understanding and Tuning Memory — Node.js Learn”. Node.js — Run JavaScript Everywhere. Accedido el 8 de octubre de 2026. [En línea]. Disponible: <https://nodejs.org/learn/diagnostics/memory/understanding-and-tuning-memory>
- 4 “Trash talk: the Orinoco garbage collector · V8”. V8 JavaScript engine. Accedido el 8 de octubre de 2026. [En línea]. Disponible: <https://v8.dev/blog/trash-talk>
- 5 King. “Memory Leaks in JavaScript: How to Detect and Fix Them”. SHIFT ASIA — Dev Blog. Accedido el 8 de octubre de 2026. [En línea]. Disponible: <https://shiftasia.com/community/memory-leaks-in-javascript-how-to-detect-and-fix-them/>